

MODUL 4

State Hoisting, Perubahan Konfigurasi, dan Side-effect

Nama: Ahsana Maswa

NIM: 245150607111008

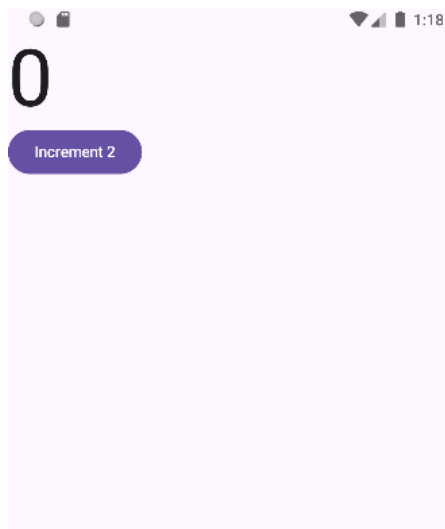
Percobaan

State Hoisting

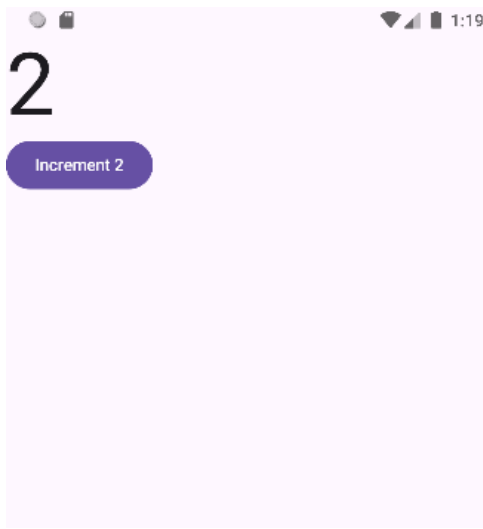
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.
 - a. Apa yang terjadi setiap kali tombol "Increment" diklik?
Angka yang ditampilkan pada layar (Composable Text) akan bertambah 1 setiap kali tombol diklik (misal dari 0, 1, 2, 3, dst).
 - b. Apa yang menyebabkan hal tersebut terjadi?
Ini terjadi karena variabel number dideklarasikan menggunakan remember { mutableStateOf(0) }, yang membuat Compose melacak perubahan nilainya. Saat tombol diklik, lambda onClick = { number++ } dijalankan, mengubah nilai number. Karena number adalah state yang diobservasi oleh Compose, perubahan nilainya memicu recomposition, yaitu Compose menggambar ulang (redraw) bagian UI yang bergantung pada number, dalam hal ini Composable Text.
 - c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
Variabel number (bertipe Int, dibungkus mutableStateOf(0)).
 - d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
Variabel label (bertipe String, dibungkus mutableStateOf("Increment")).
2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".
 - a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?
Ubah lambda onClick dari { number++ } menjadi { number += 2 } (atau { number = number + 2 }). Ubah nilai awal label dari "Increment" menjadi "Increment 2".
 - b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Sebelum:



Sesudah:



3. Langkah selanjutnya adalah melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable `CounterScreen`. Ubah signature kode program pada bagian fungsi Composable `CounterScreen` dari yang sebelumnya:

```
fun CounterScreen(modifier: Modifier)
```

Menjadi:

```
fun CounterScreen(modifier: Modifier
```

```
number: Int, label: String, onClick: () -> Unit)
```

Ubah *body* di dalam Composable `CounterScreen` menjadi:

```
// var number by remember { mutableStateOf(0) }
// val label by remember { mutableStateOf("Increment") }
Column(modifier) {
    Text(text = "$number", fontSize = 72.sp)
    Button(onClick = onClick) {
        Text(text = "$label")
    }
}
```

Ubah pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

```
var number by remember { mutableStateOf(0) }
CounterScreen(Modifier.padding(32.dp),
    number = number,
    label = "Increment",
    onClick = { number++ })
```

)

Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().

4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, secara tampilan dan perilaku, hasilnya identik. angka tetap bertambah 1 setiap klik, label tetap "Increment". Pengguna tidak akan melihat perbedaan apa pun dari sisi UI.

- b. Di bagian kode program manakah saat ini proses increment dilakukan?

Proses increment (number++) sekarang dilakukan di dalam fungsi onCreate(), tepatnya di dalam lambda onClick = { number++ } saat memanggil CounterScreen. Bukan lagi di dalam fungsi CounterScreen itu sendiri.

- c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State number dideklarasikan di onCreate() (var number by remember { mutableStateOf(0) }), sedangkan label sekarang bukan lagi state — ia hanya parameter String biasa (label = "Increment") yang dioper langsung sebagai nilai tetap (literal), tidak lagi dibungkus remember/mutableStateOf.

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

- a. Label tombol "Increment" menjadi "Double"

- b. Nilai awal variabel `number` dari 0 menjadi 1.

- c. Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.

- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

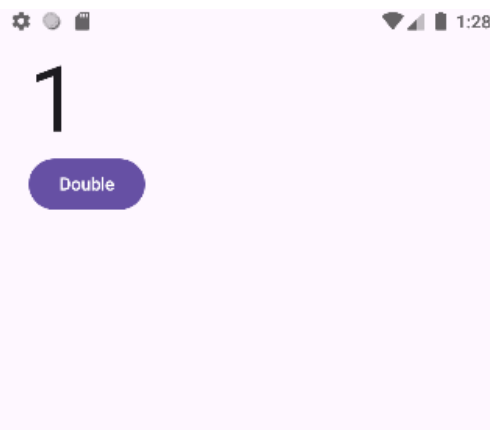
```
var number by remember { mutableStateOf(1) }
```

```
CounterScreen(  
    Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```

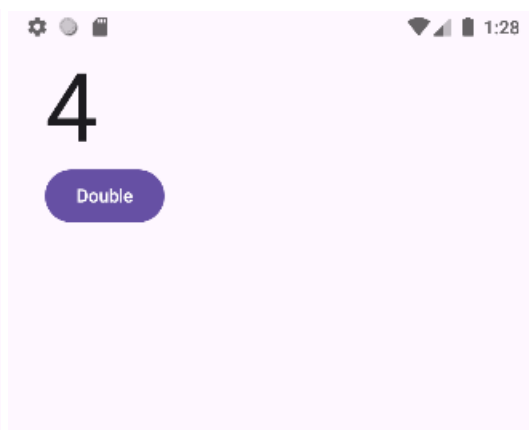
6. Jalankan program,

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Sebelum:



Sesudah klik 2x:



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Seluruh perubahan (label jadi "Double", nilai awal jadi 1, logika number $\times 2$) dilakukan di dalam fungsi onCreate(), tepatnya pada deklarasi `var number by remember { mutableStateOf(1) }` dan pada pemanggilan `CounterScreen(...)` dengan parameter label dan `onButtonClick` yang baru.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada sama sekali. Fungsi CounterScreen tidak disentuh/diubah — kode di dalamnya (yang menerima number, label, onButtonClick sebagai parameter dan menampilkannya) tetap persis sama seperti sebelumnya.

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Manfaat utama state hoisting dalam konteks perubahan kebutuhan fungsional:

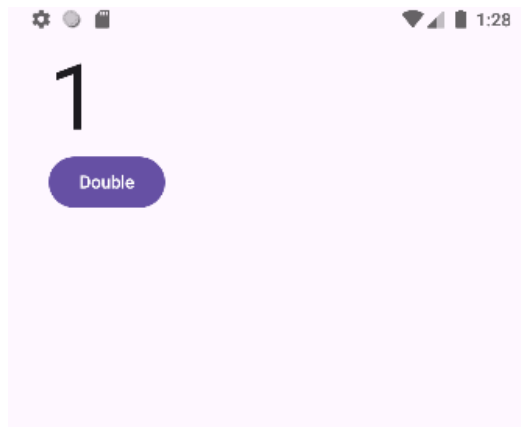
- Composable menjadi reusable (dapat digunakan ulang): CounterScreen tidak lagi menyimpan logika/state sendiri (stateless), composable yang sama bisa dipakai di berbagai konteks berbeda hanya dengan mengubah parameter yang dioper, tanpa perlu mengubah kode CounterScreen itu sendiri.
- Single source of truth: State number hanya ada di satu tempat (onCreate), sehingga lebih mudah dilacak dan dikelola, menghindari duplikasi atau state yang tidak sinkron.
- Separation of concerns (pemisahan tanggung jawab): CounterScreen murni bertanggung jawab atas tampilan (UI), sedangkan logika bisnis (bagaimana number berubah) dipindahkan ke level yang lebih tinggi
- Fleksibilitas terhadap perubahan requirement: Terbukti dari eksperimen ini, ketika kebutuhan berubah (label jadi "Double", logika jadi perkalian), tidak ada satu baris pun kode di dalam CounterScreen yang perlu diubah.

Perubahan Konfigurasi

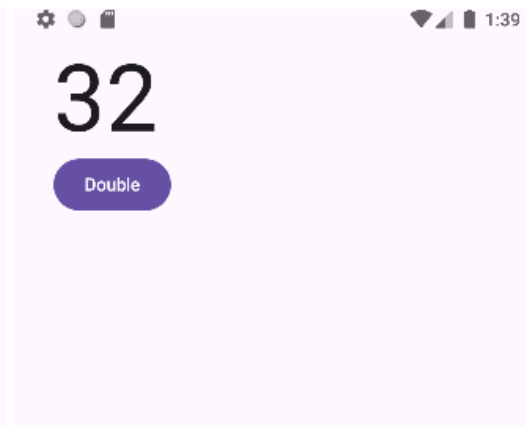
Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

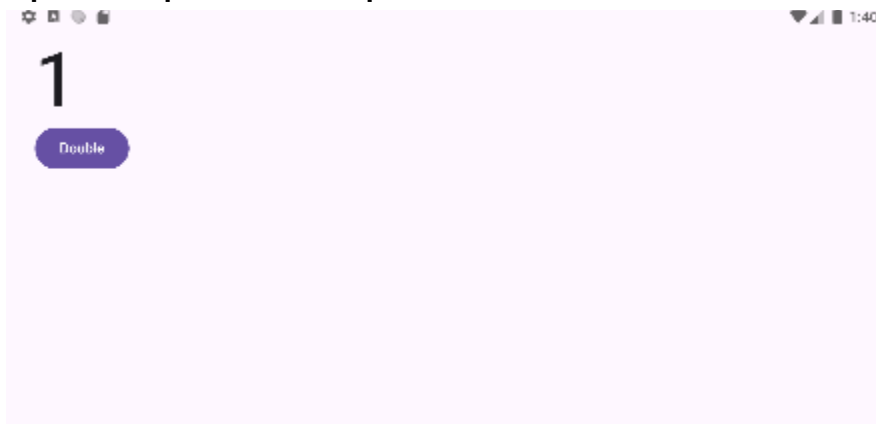
Sebelum:



Sesudah klik 5x:



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka akan kembali ke nilai awal, yaitu 1, bukan angka terakhir yang sudah di-double sebelumnya. Penyebabnya adalah perubahan orientasi layar (portrait ke landscape) adalah salah satu bentuk configuration change. Secara default, saat terjadi configuration change, sistem Android akan menghancurkan (destroy) dan membuat ulang (recreate) Activity sepenuhnya, termasuk memanggil ulang onCreate(). Karena number dideklarasikan dengan remember { mutableStateOf(1) } yang hanya bertahan selama composition masih "hidup" di memori, saat Activity di-recreate, seluruh state

yang disimpan dengan `remember` biasa akan hilang dan diinisialisasi ulang ke nilai awal (1).

Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

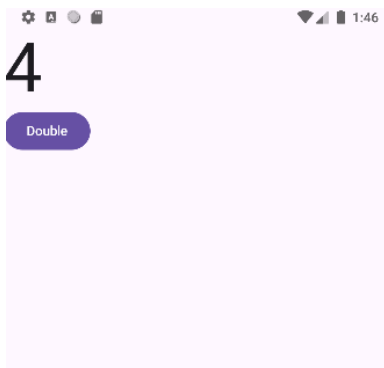
menjadi:

```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan *import dependency* atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela *editor* Android Studio).

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi *landscape*.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Kali ini angka tetap sama dengan nilai terakhir sebelum perangkat diputar (tidak kembali ke 1). Penyebabnya adalah `rememberSaveable` tidak hanya menyimpan state di memori composition seperti `remember`, tetapi juga menyimpan nilainya ke dalam Bundle (`savedInstanceState`) milik Activity. Saat terjadi configuration change dan Activity di-recreate, sistem akan memulihkan (restore) nilai number dari Bundle tersebut, sehingga nilai yang ditampilkan tetap konsisten dengan kondisi sebelum perubahan konfigurasi terjadi.

c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

remember	rememberSaveable
Menyimpan state hanya selama composition hidup di memori (selama Activity belum di-destroy)	Menyimpan state ke Bundle (mirip <code>onSaveInstanceState</code>), sehingga bertahan melewati configuration change (rotasi layar) dan bahkan proses process death oleh sistem (misal karena kekurangan memori saat di-background)
State akan hilang (reset ke nilai awal) saat Activity di-recreate	State akan dipulihkan (restored) ke nilai terakhir saat Activity di-recreate
Cocok untuk state sementara yang tidak masalah jika hilang	Cocok untuk state penting yang harus dipertahankan dari sudut pandang pengguna (misal: input form, counter, posisi scroll)

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh: Form input teks pada aplikasi, misalnya kolom isian komentar atau draft pesan chat yang sedang diketik pengguna.

Manfaat: Jika pengguna sedang mengetik komentar panjang, lalu secara tidak sengaja memutar layar perangkat (portrait ke landscape), dengan `rememberSaveable`, teks yang sudah diketik tidak akan hilang, pengguna tidak perlu mengetik ulang dari awal. Ini sangat meningkatkan pengalaman pengguna (UX), karena tanpa `rememberSaveable` (hanya pakai `remember` biasa), teks yang sudah diketik akan hilang total setiap kali terjadi configuration change, yang tentu sangat mengganggu dan membuat frustrasi pengguna.

LaunchedEffect

Aktivitas Percobaan 1

1. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Muncul Snackbar (notifikasi kecil di bagian bawah layar) bertuliskan "Number 1 is shown!" (karena nilai awal number adalah 1, hasil dari `rememberSaveable { mutableStateOf(1) }`).

- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena `LaunchedEffect(number)` otomatis dijalankan setiap kali `Composable` tersebut pertama kali memasuki `composition` (muncul di layar untuk pertama kalinya), terlepas dari ada tidaknya interaksi pengguna. `LaunchedEffect` dirancang khusus untuk menjalankan kode `suspend/efek samping` (side-effect) yang terikat pada siklus hidup `composition`, begitu `composable` yang memanggilnya pertama kali di-`compose`, blok kode di dalamnya langsung dieksekusi.

- c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?

Yang memicu adalah pemanggilan `LaunchedEffect(number) { snackbarHostState.showSnackBar(...) }`. Secara spesifik, `LaunchedEffect` akan otomatis ter-trigger kembali setiap kali nilai `number` (sebagai `key`)-nya berubah, karena `Compose` membandingkan `key` tersebut antar `recomposition`, jika berbeda dari sebelumnya, `coroutine` lama akan dibatalkan dan blok efeknya dijalankan ulang dengan nilai `number` yang baru.

2. Klik *button* "Double" satu kali,

- a. Apa yang terjadi?

`SnackBar` baru muncul lagi, kali ini dengan pesan "Number 2 is shown!" (karena `number` sekarang berubah dari 1 menjadi 2 akibat digandakan).

- b. Mengapa hal tersebut terjadi? Jelaskan!

Karena `number` digunakan sebagai `key` pada `LaunchedEffect(number)`. Ketika tombol diklik, nilai `number` berubah (akibat `state hoisting` sebelumnya, `onButtonClick` memicu perkalian nilai `number`), sehingga terjadi `recomposition`. `Compose` mendeteksi bahwa `key` (`number`) pada `LaunchedEffect` sudah berbeda dari komposisi sebelumnya, sehingga ia membatalkan `coroutine` lama dan menjalankan ulang blok `LaunchedEffect` dengan nilai `number` yang baru, memicu munculnya `SnackBar` baru dengan pesan yang sudah diperbarui.

Aktivitas Percobaan 2

1. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Awalnya layar menampilkan angka 1 (nilai awal number) beserta Snackbar "Number 1 is shown!" sama seperti sebelumnya, sesaat sebelum proses pengambilan data simulasi selesai.

b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Setelah kurang lebih 2 detik (sesuai `delay(2000)` di `Repo.getData()`), angka pada layar berubah menjadi angka acak hasil dari `Random.nextInt(100, 1000)` (misalnya 457, 823, dll, nilainya acak setiap kali dijalankan). Snackbar baru juga muncul menampilkan pesan sesuai angka acak tersebut, misalnya "Number 457 is shown!".

c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Dipicu oleh `LaunchedEffect(Unit) { number = Repo.getData() }`. Fungsi `Repo.getData()` adalah fungsi suspend yang menunda eksekusi selama 2 detik (`delay(2000)`), lalu mengembalikan angka acak. Nilai kembaliannya langsung menggantikan nilai `number`. Karena `number` adalah state yang diobservasi, perubahan nilainya memicu recomposition, yang pada gilirannya juga memicu `LaunchedEffect(number)` (yang pertama) untuk menampilkan Snackbar baru dengan angka terbaru.

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

`LaunchedEffect(state)`:

- Blok kode di dalamnya akan dijalankan ulang setiap kali nilai key (`number`) berubah di antara proses recomposition
- Cocok digunakan ketika efek samping perlu bereaksi terhadap perubahan data tertentu (misalnya menampilkan notifikasi setiap kali counter berubah)

`LaunchedEffect(Unit)`:

- Blok kode di dalamnya hanya dijalankan satu kali saja, yaitu saat composable pertama kali memasuki composition, karena `Unit` adalah nilai yang tidak pernah berubah
- Cocok digunakan untuk operasi inisialisasi satu kali yang hanya perlu dijalankan sekali saat composable pertama kali muncul (misalnya memuat data awal dari API/database saat layar pertama kali dibuka)

b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

- `LaunchedEffect(state)`: Dijalankan pertama kali saat composable memasuki composition, dan akan dijalankan ulang (coroutine lama dibatalkan, lalu dijalankan ulang dari awal) setiap kali nilai state yang menjadi key-nya berubah pada recomposition berikutnya.
- `LaunchedEffect(Unit)`: Dijalankan hanya satu kali, yaitu saat composable pertama kali memasuki composition (enters composition), dan tidak akan pernah dijalankan ulang selama composable tersebut tetap berada dalam composition, karena `Unit` secara definisi selalu bernilai sama (tidak ada variasi nilai yang bisa dibandingkan untuk memicu re-run), tidak peduli berapa kali terjadi recomposition akibat state lain yang berubah.